

Trabajo Práctico Parte 2 — Informe

Introducción a la Bioinformática — UTN FRBA Fecha de entrega: 26 de junio de 2026

Esta segunda parte continúa el trabajo sobre el gen **HTT** (Huntingtin) y la enfermedad de Huntington iniciado en la Parte 1. Acá se desarrollan los ejercicios 4 (parser de la salida de BLAST), 5 (análisis con EMBOSS) y 6 (trabajo con bases de datos biológicas). El ambiente de trabajo es el mismo de la Parte 1: Python 3.11 con BioPython, EMBOSS y BLAST+ dentro de un contenedor Docker, con el código versionado en Git.

El **Ejercicio 6** se entrega como documento aparte (`ex6_bases_de_datos.md` / su PDF), porque incluye numerosas capturas de pantalla de las bases de datos consultadas.

1. Ejercicio 4 — Parser de la salida de BLAST

Descripción

Se desarrolló el script `src/ex4_blast_parser.py`, que toma como entrada el reporte `output/blast.out` generado en el Ejercicio 2 y un **Pattern** pasado como parámetro por línea de comandos (por defecto `"Mus musculus"`, como en el ejemplo de la consigna). El script recorre el reporte e identifica los hits cuya descripción contiene ese Pattern, de forma **case-insensitive** (para que `"mus musculus"` y `"Mus musculus"` coincidan).

Para distinguir las filas de hit de las líneas decorativas del reporte se usa una expresión regular que detecta el **ACCESSION** al inicio de cada fila (formato tipo `P42859.2`). Además, mientras recorre el archivo, el script va guardando el frame actual (`Frame +2`, etc.) para poder indicar en qué marco de lectura apareció cada hit.

Punto extra: para cada hit que coincide con el Pattern, el script extrae su **ACCESSION** y descarga la secuencia completa de la proteína en formato FASTA desde NCBI usando `Bio.Entrez` (el equivalente en BioPython al módulo `Bio::DB::GenBank` de BioPerl que menciona la consigna), y la escribe a `output/ex4_hits.fasta`.

```
docker compose run tp python src/ex4_blast_parser.py "Mus musculus"
# input:  output/blast.out + Pattern
# output: output/ex4_hits.fasta
```

Resultados

Ejecutando con el Pattern `"Mus musculus"` sobre el `blast.out` del Ejercicio 2, el script encontró **1 hit** que coincide:

Frame	Accession	Descripción
+2	P42859.2	Huntingtin — <i>Mus musculus</i> (ratón)

A continuación descargó la secuencia completa de ese hit desde NCBI y la guardó en `output/ex4_hits.fasta`:

```
>sp|P42859.2|HD_MOUSE RecName: Full=Huntingtin; AltName: Full=Huntington
disease protein homolog; Short=HD protein homolog; ...
```

Si se ejecuta con otro Pattern (por ejemplo `"Rattus"` o `"Takifugu"`) el script devuelve los hits correspondientes a esa otra especie, lo que confirma que el filtro funciona de forma genérica.

Conclusión del ejercicio

El ejercicio muestra cómo procesar de forma programática la salida de una herramienta como BLAST en lugar de leerla a mano. Lo más interesante fue el punto extra: con solo el ACCESSION del hit se puede recuperar automáticamente la secuencia original completa desde NCBI, encadenando el resultado del BLAST con una nueva consulta a la base de datos. Esto deja todo listo para análisis posteriores (por ejemplo, un alineamiento) sin pasos manuales.

2. Ejercicio 5 — EMBOSS (ORFs y dominios PROSITE)

Descripción

Se desarrolló el script `src/ex5_emboss.py`, que arma un pequeño pipeline llamando a varios programas del paquete **EMBOSS** sobre el mRNA de HTT (`data/NM_002111.gb`):

1. **seqret** convierte el mRNA del formato GenBank a FASTA de nucleótidos.
2. **getorf** calcula los ORFs (marcos de lectura abiertos) del mRNA y los traduce a proteína. Se usó `-find 1` (traducir las regiones entre un codón de inicio ATG y el de stop) y `-minsize 300` (300 nucleótidos = 100 aa, para descartar ORFs cortos espurios).
3. El script selecciona el **ORF más largo**, que corresponde a la huntingtina.
4. Descarga la base de datos **PROSITE** (`prosite.dat` + `prosite.doc`) desde el FTP de ExPASy y la indexa con **prosextract**.
5. Finalmente, **patmatmotifs** analiza los dominios/motivos funcionales conocidos de la proteína contra PROSITE y escribe el resultado en `output/ex5_domains.txt`.

```
docker compose run tp python src/ex5_emboss.py
# input:  data/NM_002111.gb
# output: output/ex5_orfs.fasta, output/ex5_longest_orf.fasta, output/ex5_domains.txt
```

Resultados

getorf encontró **8 ORFs** de al menos 100 aa. El más largo es la huntingtina:

```
>NM_002111_1 [146 - 9577] Homo sapiens huntingtin (HTT), transcript variant 2, mRNA
Longitud: 3144 aa
```

Este tamaño (3144 aa) coincide con el de la huntingtina humana conocida y con el frame +2 que ya habíamos identificado en la Parte 1.

El análisis de dominios con **patmatmotifs** contra PROSITE encontró **4 motivos funcionales**:

Motivo	Posición (aa)	Qué es
AMIDATION	1532–1535	Sitio de amidación
AMIDATION	2545–2548	Sitio de amidación
LEUCINE_ZIPPER	1446–1467	Patrón "cremallera de leucina"
TYR_PHOSPHO_SITE_2	2716–2723	Sitio de fosforilación de tirosina

Conclusión del ejercicio

Lo más útil de este ejercicio fue ver cómo se pueden encadenar varias herramientas de EMBOSS en un mismo pipeline: a partir del mRNA en GenBank se obtienen los ORFs, se elige automáticamente el más largo y sobre esa proteína se buscan dominios funcionales contra una base de datos externa (PROSITE). Que `getorf` recupere la huntingtina completa de 3144 aa de forma independiente confirma el resultado de la Parte 1 por otro camino. Los motivos PROSITE encontrados (sitios de fosforilación, amidación y una cremallera de leucina) dan una primera pista de las regiones funcionales de la proteína.

3. Ejercicio 6 — Bases de datos biológicas

Este ejercicio se entrega como **documento separado** (`ex6_bases_de_datos.md` y su PDF), donde se recorren las bases de datos NCBI Gene, Ensembl, NCBI Orthologs, UniProt, KEGG, ClinVar y MedlinePlus para describir el gen HTT, sus homólogos, transcritos, interacciones, ontología (GO), vías metabólicas y la variante patogénica (expansión CAG). Incluye las capturas de pantalla de cada consulta.

Anexo — Cómo ejecutar los scripts

Requisitos: Docker Desktop instalado y corriendo.

```
# Entrar al contenedor
docker compose run tp bash

# Ejercicios de la Parte 2
python src/ex4_blast_parser.py "Mus musculus" # Genera output/ex4_hits.fasta
python src/ex5_emboss.py                      # Genera output/ex5_*.fasta y
ex5_domains.txt
```